VULNERABILITY ASSESSMENT REPORT - TASK 1

1. Introduction

This report presents the results of a vulnerability assessment conducted on the following web application:

Target Application: DVWA (Damn Vulnerable Web Application)
Target URL: http://localhost:8080

The purpose of this assessment is to identify common web application vulnerabilities in a controlled environment and provide actionable recommendations to improve the system's overall security posture.

2. Objectives

• Identify common web application vulnerabilities
• Perform basic security testing on application inputs
• Classify vulnerabilities based on severity (High/Medium/Low)
• Provide remediation strategies in simple, business-friendly language

3. Tools Used

• Web Browser: Manual testing of application behavior
• Browser Developer Tools: Request/response analysis
• Docker: Deployment of test environment
• Manual Testing Techniques

4. Methodology

1. The DVWA application was deployed in a local environment using Docker
2. Security level was configured to "Low" to simulate vulnerable conditions
3. Different input fields were tested for improper validation
4. Payloads were injected to identify vulnerabilities
5. Findings were analyzed and categorized based on risk severity

5. Vulnerability Findings

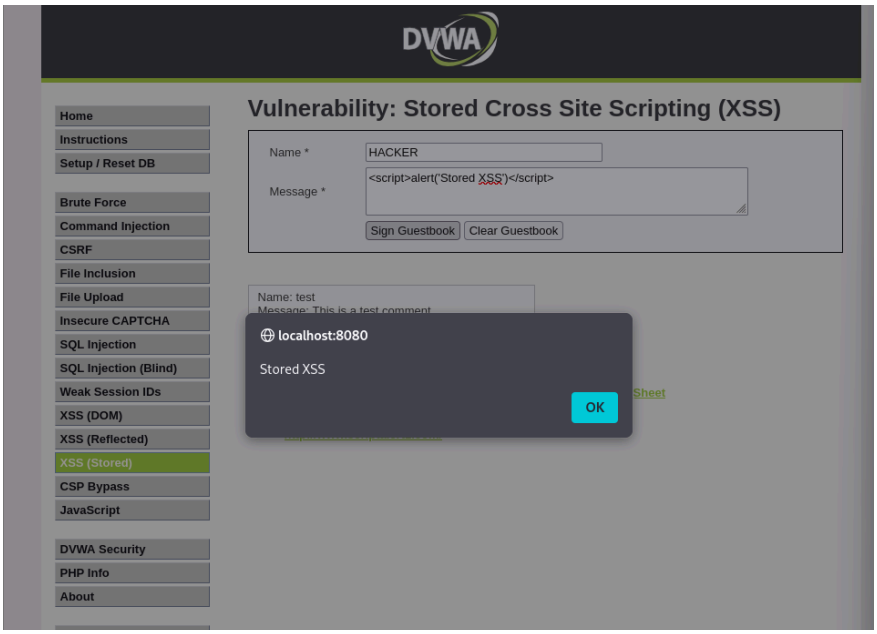
5.1 Stored Cross-Site Scripting (Stored XSS)

• **Risk Level:** Critical

Description:
The application allows users to input malicious JavaScript code into the guestbook, which is stored in the database and executed whenever the page is loaded.

Impact:
• Affects all users accessing the page
• Enables persistent malicious script execution
• Can lead to session hijacking and data theft

Proof of Concept:
Payload used: <script>alert('Stored XSS')</script>



Recommendation:
• Sanitize all user inputs
• Encode output before rendering
• Implement Content Security Policy (CSP)

5.2 Reflected Cross-Site Scripting (XSS)

• **Risk Level:** High

Description:

Reflected Cross-Site Scripting occurs when user-supplied input is immediately returned in the web application's response without proper validation or sanitization. In this case, the application reflects the input directly into the page, allowing execution of arbitrary JavaScript code in the user's browser.

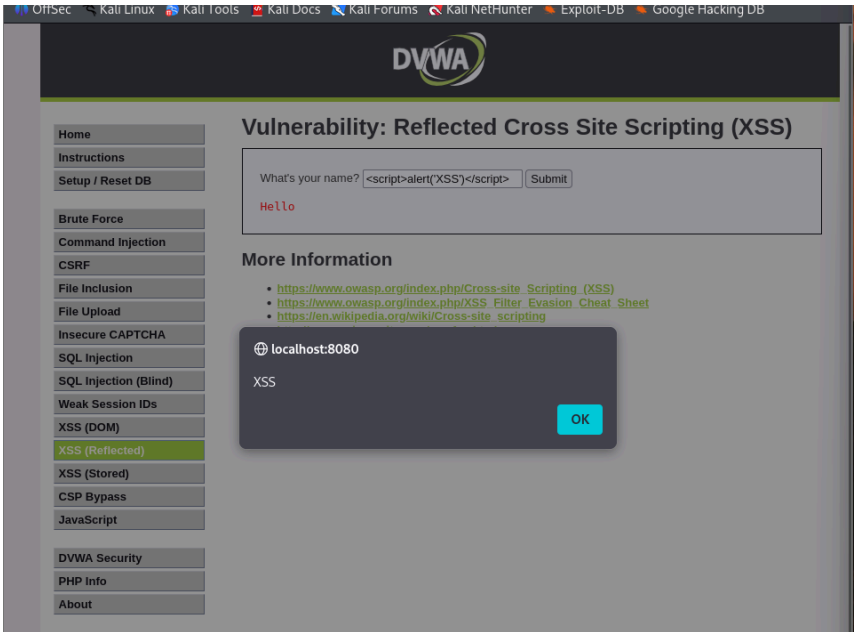
Impact:

• Execution of malicious JavaScript in the victim's browser
• Theft of session cookies and sensitive information
• Redirection to malicious or phishing websites
• Defacement of web pages

Proof of Concept:

Payload used: <script>alert('XSS')</script>

The payload successfully triggered a JavaScript alert, confirming that the application does not properly sanitize user input.



Recommendation:

• Validate and sanitize all user inputs
• Encode output before displaying

• Implement security headers.

5.3 Weak Password Policy

• **Risk Level:** High

Description:

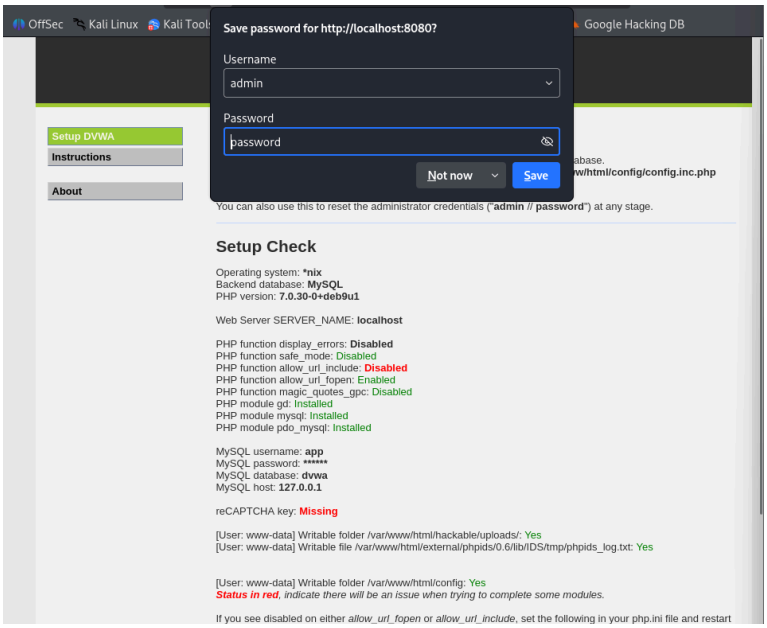
The application allows users to create accounts using weak and easily guessable passwords such as “1234”. There are no enforced password complexity requirements, making it easier for attackers to compromise user accounts.

Impact:

• Increased risk of brute-force and dictionary attacks
• Unauthorized access to user accounts
• Potential data breaches and misuse of sensitive information

Proof of Concept:

A user account was successfully created using the weak password: password



Recommendation:

• Enforce minimum password length (at least 8–12 characters)
• Require a combination of uppercase, lowercase, numbers, and special characters
• Implement password strength indicators
• Enable Multi-Factor Authentication (MFA)

• Apply account lockout after multiple failed attempts

5.4 SQL Injection (Tested)

• **Risk Level:** Medium

Description:

SQL Injection vulnerabilities occur when user input is directly incorporated into SQL queries without proper validation or sanitization. During testing, SQL payloads were injected into input fields, and the application returned SQL-related errors, indicating improper handling of input.

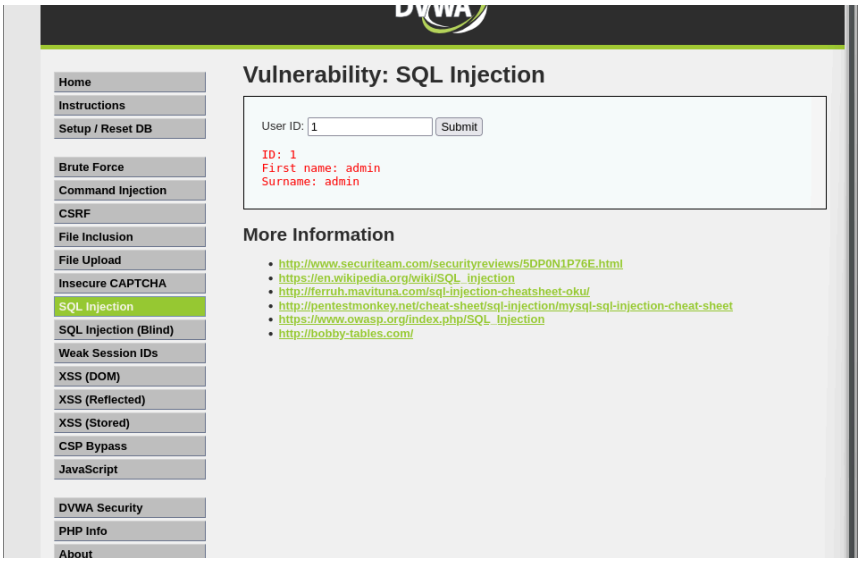
Impact:

• Unauthorized access to database contents
• Exposure of sensitive information such as user credentials
• Modification or deletion of database records
• Potential complete database compromise

Proof of Concept:

Payload tested: 1 OR 1=1

The application response indicated improper input handling, suggesting potential vulnerability to SQL Injection.



Recommendation:

• Use prepared statements (parameterized queries)
• Avoid dynamic SQL queries
• Implement strict input validation and sanitization

• Use ORM frameworks that prevent injection attacks
• Limit database user privileges.

6. Risk Summary

Risk Level	Count
Critical	1
High	2
Medium	1
Low	0
Informational	0

Analysis:
The majority of identified vulnerabilities fall under High and Critical risk categories, indicating significant weaknesses in input validation and authentication mechanisms. Immediate remediation is recommended to prevent potential exploitation.

7. Conclusion

The vulnerability assessment revealed multiple security weaknesses within the application, primarily due to improper input validation, lack of output encoding, and weak authentication controls.

Critical vulnerabilities such as Stored Cross-Site Scripting demonstrate the risk of persistent attacks that can affect all users of the application. Additionally, reflected XSS and weak password policies increase the likelihood of client-side attacks and unauthorized account access.

Although the application was tested in a controlled environment, similar vulnerabilities in real-world systems could lead to serious security breaches,

including data theft, session hijacking, and system compromise.

8. Recommendations (Final Summary)

• Implement strong input validation and output encoding across all application layers
• Enforce secure authentication mechanisms, including strong password policies and MFA
• Apply essential security headers such as Content Security Policy (CSP) and X-Frame-Options
• Use prepared statements and secure coding practices to prevent SQL Injection
• Regularly perform vulnerability assessments and penetration testing
• Follow industry best practices such as OWASP Top 10 guidelines
• Conduct security awareness training for developers and stakeholders